



INFORME

I. PORTADA

Tema:	Recorridos Inorden, Preorden, Postorden y BFS
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero “B”
Alumnos participantes:	Moyolema Criollo Katherine Lissette
Asignatura:	Estructura de datos
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Implementar y analizar los principales recorridos de árboles binarios utilizando C++ y Java, aplicando estructuras de datos dinámicas, recursividad y colas.

Específicos:

- Implementar los recorridos Inorden, Preorden y Postorden en C++ y Java utilizando recursividad, verificando la correcta visita de los nodos en cada orden
- Aplicar el recorrido BFS mediante el uso de una cola dinámica, comparando su funcionamiento con los recorridos en profundidad (DFS).
- Analizar y comparar los resultados obtenidos en ambos lenguajes de programación (C++ y Java), relacionándolos con el caso aplicado del sistema web para demostrar la utilidad de los recorridos en la organización de módulos.

2.2 Introducción

Los árboles binarios son estructuras de datos fundamentales en informática que permiten organizar información de forma jerárquica y eficiente, facilitando operaciones como búsqueda, inserción y eliminación. Sus recorridos principales —Inorden, Preorden, Postorden y BFS— permiten procesar los nodos de distintas maneras según la necesidad, como ordenar datos, evaluar expresiones o recorrer niveles. El estudio de estos recorridos es clave para comprender el manejo de estructuras jerárquicas en programación y su aplicación en sistemas reales.

En esta práctica, desarrollada en modalidad grupal con GitHub, se implementaron los recorridos en **C++ y Java**, aplicando recursividad y colas dinámicas. Además, se relacionó el aprendizaje con un caso aplicado: la representación de los módulos de un sistema web como un árbol binario, lo que permite visualizar jerarquías como gestión de usuarios e inventario, y aplicar los recorridos para mostrar menús, procesar operaciones internas o recorrer niveles.

2.3 Justificación

La práctica de recorridos de árboles binarios se justifica porque constituye un eje fundamental en la formación de un ingeniero de software. Comprender y aplicar los recorridos Inorden, Preorden, Postorden y BFS permite desarrollar competencias esenciales en el manejo de estructuras jerárquicas, las cuales son ampliamente utilizadas en sistemas de archivos, bases de datos y aplicaciones de inteligencia artificial [1].

La implementación en C++ y Java fortalece la capacidad de análisis comparativo entre lenguajes de programación, promoviendo el dominio de técnicas como la recursividad y el uso de colas dinámicas, que son pilares en la resolución de problemas algorítmicos [2][3].



Asimismo, el caso aplicado de representar los módulos de un sistema web como un árbol binario conecta la teoría con la práctica, evidenciando cómo los recorridos pueden utilizarse para organizar jerarquías de usuarios, inventario y operaciones internas. Este enfoque demuestra la utilidad de los árboles binarios más allá del ámbito académico, vinculándolos con problemas reales de diseño y modelado de software [7].

Finalmente, el trabajo grupal con GitHub fomenta competencias de colaboración, documentación y uso de herramientas modernas de control de versiones, aspectos indispensables en el desarrollo profesional de proyectos de software.

2.4 Marco teórico

2.4.1 Definición de árboles binarios

Un árbol binario es una estructura de datos jerárquica compuesta por nodos, donde cada nodo puede tener como máximo dos hijos: izquierdo y derecho. El nodo principal se denomina raíz, mientras que los nodos sin hijos se conocen como hojas. Además, cada nodo puede considerarse la raíz de un subárbol, lo que evidencia la naturaleza recursiva de esta estructura.

Entre sus propiedades básicas destacan la altura, que representa el número de niveles del árbol; la profundidad, que indica la distancia desde la raíz hasta un nodo específico; y el balance, que se refiere a la distribución equilibrada de los nodos, lo cual influye directamente en la eficiencia de las operaciones [1] [4].

2.4.2 Recorridos de árboles binarios

Los recorridos permiten visitar sistemáticamente todos los nodos del árbol:

- **Inorden** (Izquierda → Raíz → Derecha): Permite obtener los elementos en orden ascendente en árboles binarios de búsqueda (BST).
- **Preorden** (Raíz → Izquierda → Derecha): Se utiliza en la construcción de expresiones y representación de estructuras jerárquicas como menús.
- **Postorden** (Izquierda → Derecha → Raíz): Es útil en la liberación de memoria y en el procesamiento de estructuras internas.
- **BFS** (Breadth-First Search): Recorre el árbol nivel por nivel utilizando una cola, siendo ideal para sistemas jerárquicos y exploración de grafos[6][8].

2.4.3 Estructuras de datos utilizadas

Los recorridos en profundidad (Inorden, Preorden y Postorden) se implementan comúnmente mediante recursividad, aprovechando la naturaleza jerárquica del árbol. Por otro lado, el recorrido BFS requiere el uso de colas dinámicas, ya que permite procesar los nodos en orden de niveles.

2.4.4 Aplicaciones reales

Los árboles binarios tienen múltiples aplicaciones en informática, entre ellas:

- **Sistemas de archivos:** Representación de directorios y subdirectorios.
- **Bases de datos:** Uso en estructuras como árboles de búsqueda binaria y árboles B.
- **Compiladores:** Construcción de árboles de sintaxis para analizar expresiones.
- **Sistemas web:** Organización de módulos, menús y jerarquías funcionales [1][3][5].



2.5 Marco Metodológico

El marco metodológico de la práctica se basó en un trabajo grupal apoyado en GitHub, lo que permitió la colaboración, control de versiones y documentación del proceso. Se utilizaron los lenguajes de programación C++ y Java para implementar los recorridos de árboles binarios, aplicando recursividad en los recorridos en profundidad y colas dinámicas en el recorrido BFS.

El procedimiento incluyó la creación de un repositorio en GitHub, la copia de la estructura del proyecto base, la ejecución del código en ambos lenguajes, la comparación de resultados, la inserción de nodos adicionales y la documentación en el archivo README.md junto con capturas de ejecución.

Como instrumentos se emplearon el código fuente, la guía práctica, los ejercicios, el banco de preguntas Moodle y el README del grupo. Finalmente, se aplicó el conocimiento en un caso real: la representación de los módulos de un sistema web como árbol binario, lo que permitió visualizar jerarquías y operaciones internas mediante los distintos recorridos [1][4][6][8].

2.6 Desarrollo

Durante la práctica se trabajó con distintos archivos que permitieron aplicar los conceptos teóricos de los recorridos de árboles binarios en C++ y Java, siguiendo la guía metodológica y el caso aplicado del sistema web.

2.6.1 Implementación de recorridos base (C++ y Java)

Dentro de la carpeta src/ se encuentran dos carpetas, en las que hay archivos con código tanto en C++ y Java.

Se implementaron los cuatro recorridos en ambos lenguajes, añadiendo como mínimo 5 nodos más, aparte de las que ya están en el código base:

- Código en C++

En C++ se utilizó una estructura struct Nodo con punteros para manejar la memoria dinámica. La lógica de los recorridos es la siguiente:

```
/**
 * RECORRIDO PREORDEN (Raíz -> Izquierda -> Derecha)
 * Útil para: crear copias del árbol, mostrar estructura jerárquica
 * Complejidad: O(n) donde n es número de nodos
 */
void preorden(Nodo* raiz) {
    if (raiz == nullptr) return;          // Caso base: nodo vacío

    cout << raiz->dato << " ";           // 1. Visitar raíz
    preorden(raiz->izquierda);             // 2. Recorrer izquierda (recursivo)
    preorden(raiz->derecha);               // 3. Recorrer derecha (recursivo)
}
```

Explicación de la lógica:

- **Preorden:** Primero se visita la raíz, luego se recorre recursivamente el subárbol izquierdo y finalmente el derecho. Útil para crear copias del árbol.
- **Inorden:** Se recorre primero el subárbol izquierdo, luego se visita la raíz y finalmente el derecho. En un BST produce valores ordenados.
- **Postorden:** Se recorren ambos subárboles antes de visitar la raíz. Útil para eliminar árboles.
- **BFS:** Utiliza una cola FIFO para procesar los nodos nivel por nivel, comenzando desde la raíz.

- Código Java



En Java se utilizó una clase Nodo con referencias y la interfaz Queue implementada con LinkedList. La lógica es equivalente a C++ pero con sintaxis propia del lenguaje:

```
/**
 * RECORRIDO BFS (Breadth-First Search) o NIVEL POR NIVEL
 * Útil para: encontrar el camino más corto, mostrar niveles
 * Estructura utilizada: COLA (FIFO - First In First Out)
 * Diferencia con DFS: recorre por niveles, no por profundidad
 *
 * @param raiz Nodo raíz del árbol
 */
public static void bfs(Nodo raiz) {
    if (raiz == null) return;

    // Usamos LinkedList como implementación de Queue (cola)
    Queue<Nodo> cola = new LinkedList<>();
    cola.add(raiz); // Comenzamos con la raíz

    while (!cola.isEmpty()) {
        Nodo actual = cola.poll(); // Obtener y eliminar el primer nodo

        System.out.print(actual.dato + " "); // Visitar el nodo actual

        // Agregar los hijos a la cola (si existen)
        if (actual.izquierda != null)
            cola.add(actual.izquierda);
        if (actual.derecha != null)
            cola.add(actual.derecha);
    }
}
```

2.6.2 Ejercicios resueltos

Ejercicio 1: Recorridos manuales del árbol base

Árbol base:

```
      10
     /  \
    5    15
   / \  / \
  2  7 12 20
```

Recorrido	Orden de visita	Resultado
Preorden	Raíz → Izq → Der	10, 5, 2, 7, 15, 12, 20
Inorden	Izq → Raíz → Der	2, 5, 7, 12, 15, 20
Postorden	Izq → Der → Raíz	2, 7, 5, 12, 20, 15, 10
BFS	Nivel por nivel (cola)	10, 5, 15, 2, 7, 12, 20

Estos resultados coinciden con la ejecución del programa base (sin nodos adicionales)

Ejercicio 2: Árbol ampliado (nodos 1, 3, 18, 25, 30)

```
      10 (Raíz - Nivel 0)
     /  \
    5    15 (Nivel 1)
   / \  / \
  2  7 12 20 (Nivel 2)
 / \      / \
1  3    18 25 (Nivel 3)
                \
               30 (Nivel 4)
```



Recorrido	Resultado
Preorden	10, 5, 2, 1, 3, 7, 15, 12, 20, 18, 25, 30
Inorden	1, 2, 3, 5, 7, 10, 12, 15, 18, 20, 25, 30
Postorden	1, 3, 2, 7, 5, 12, 18, 30, 25, 20, 15, 10
BFS	10, 5, 15, 2, 7, 12, 20, 1, 3, 18, 25, 30

El recorrido Inorden ahora muestra los valores completamente ordenados de menor a mayor porque el árbol mantiene propiedades de BST. El BFS muestra claramente los niveles: nivel 0 (10), nivel 1 (5,15), nivel 2 (2,7,12,20), nivel 3 (1,3,18,25), nivel 4 (30).

Ejercicio 3: Función contar nodos

Código en C++:

```
int contarNodos(Nodo* raiz) {  
    if (raiz == nullptr) return 0;  
    // Sumo: 1 (nodo actual) + nodos izquierda +  
    // nodos derecha  
    return 1 + contarNodos(raiz->izquierda) +  
    contarNodos(raiz->derecha);  
}
```

Código en Java:

```
public static int contarNodos(Nodo raiz) {  
    if (raiz == null) return 0;  
    return 1 + contarNodos(raiz.izquierda) +  
    contarNodos(raiz.derecha);  
}
```

Explicación: La función utiliza recursividad. El caso base es cuando el nodo es null (retorna 0). En cada nodo válido, suma 1 (el nodo actual) más el resultado de contar los nodos del subárbol izquierdo y derecho. Para el árbol ampliado de 12 nodos, el resultado es **12**.

Complejidad: $O(n)$ donde n es el número de nodos, porque visita cada nodo exactamente una vez.

Ejercicio 4: Función contar hojas

Código en C++:

```
int contarHojas(Nodo* raiz) {  
    if (raiz == nullptr) return 0;  
    // Si es hoja (sin hijos), retorna 1  
    if (raiz->izquierda == nullptr && raiz->derecha ==  
    nullptr)  
        return 1;  
    // Sino, suma las hojas de los subárboles  
    return contarHojas(raiz->izquierda) +  
    contarHojas(raiz->derecha);  
}
```

Código en Java:

```
public static int contarHojas(Nodo raiz) {  
    if (raiz == null) return 0;  
    if (raiz.izquierda == null && raiz.derecha ==  
    null)  
        return 1;  
    return 1 + contarHojas(raiz.izquierda) +  
    contarHojas(raiz.derecha);  
}
```



```
        return      contarHojas(raiz.izquierda)      +  
        contarHojas(raiz.derecha);  
    }
```

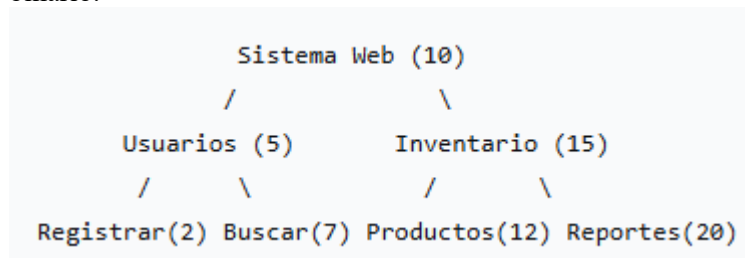
Explicación: Una hoja es un nodo que no tiene hijos izquierdo ni derecho. La función recursiva verifica esta condición: si es hoja retorna 1; si no, suma las hojas encontradas en sus subárboles izquierdo y derecho.

Para el árbol ampliado, las **5 hojas** son: 1, 3, 7, 12, 18, 30 (verificar visualmente en el árbol: nodos sin hijos)

Resultado para el árbol ampliado: 6 hojas (1, 3, 7, 12, 18, 30)

Ejercicio 5: Casa aplicado -Sistema Web

Se representó la estructura de módulos de un sistema web como un árbol binario:



Respuesta a las preguntas del ejercicio:

1. **¿Qué recorrido usaría para mostrar el menú principal?**
Preorden
2. **¿Qué recorrido usaría para procesar primero los módulos internos?**
Postorden
3. **¿Qué recorrido usaría para mostrar módulos nivel por nivel?**
BFS

Recorrido	Aplicación en el sistema web	Justificación
Preorden	Mostrar menú principal y submenús en orden jerárquico	La raíz se muestra primero (Sistema Web), luego los módulos principales (Usuarios, Inventario) y finalmente las operaciones específicas (Registrar, Buscar, Productos, Reportes)
Inorden	Mostrar elementos en orden alfabético o de prioridad	Para un menú organizado alfabéticamente (Buscar, Productos, Registrar, Reportes, Sistema Web, Usuarios) - aunque en este caso depende del orden de los nodos
Postorden	Procesar primero los módulos internos que el principal	Útil para ejecutar operaciones de limpieza o procesamiento donde los submódulos deben completarse antes que el módulo padre (ej: cerrar sesión en submódulos antes de cerrar el sistema)



BFS	Mostrar módulos nivel por nivel en interfaces web	Ideal para dashboards que muestran primero las secciones principales (nivel 1) y luego las opciones específicas (nivel 2)
-----	---	---

2.6.3 Preguntas de reflexión (Ejercicio 5 – caso aplicado: Sistema web)

- **¿Qué recorrido sirve para ordenar valores en un BST?**
El recorrido Inorden permite obtener los valores en orden ascendente. En el sistema web, si los módulos estuvieran organizados alfabéticamente, Inorden mostraría: Buscar, Inventario, Productos, Registrar, Reportes, Sistema Web, Usuarios.
- **¿Qué diferencia existe entre DFS y BFS?**
DFS (Depth-First Search) recorre en profundidad usando recursividad (pila implícita), mientras que BFS (Breadth-First Search) recorre nivel por nivel usando una cola. En el sistema web, DFS es ideal para menús jerárquicos y BFS para dashboards que muestran secciones principales primero.
- **¿Por qué BFS requiere una cola?**
Porque BFS necesita procesar los nodos en orden FIFO (First In First Out) para garantizar que los nodos del nivel actual se procesen antes que los del siguiente nivel. En el sistema web, la cola asegura que primero se muestren las secciones principales (Usuarios, Inventario) y luego las opciones específicas.
- **¿En qué caso real se puede usar Preorden?**
Preorden es ideal para mostrar un menú principal con estructura jerárquica completa, como el menú de navegación del sistema web: Sistema Web → Usuarios → Registrar/Buscar → Inventario → Productos/Reportes.
- **¿En qué caso real se puede usar Postorden?**
Postorden es útil para procesar módulos internos antes que el módulo principal, como al cerrar sesión: primero cerrar submódulos (Registrar, Buscar, Productos, Reportes), luego módulos principales (Usuarios, Inventario) y finalmente el Sistema Web.

2.6.4 Preguntas tipo Moodle (autoevaluación)

1. ¿Cuál es el orden del recorrido Inorden?

- A. Raíz, izquierda, derecha
- B. Izquierda, raíz, derecha
- C. Izquierda, derecha, raíz
- D. Nivel por nivel
- **Respuesta: B**

2. ¿Qué estructura utiliza BFS?

- A. Pila
- B. Lista circular
- C. Cola
- D. Árbol AVL
- **Respuesta: C**

3. ¿Cuál recorrido visita primero la raíz?

- A. Inorden



- B. Preorden
- C. Postorden
- D. BFS únicamente
- **Respuesta: B**

4. ¿Cuál recorrido procesa la raíz al final?

- A. Preorden
- B. Inorden
- C. Postorden
- D. Nivel por nivel
- **Respuesta: C**

5. En un BST, el recorrido Inorden permite obtener:

- A. Elementos desordenados
- B. Elementos por niveles
- C. Elementos en orden ascendente
- D. Solo hojas
- **Respuesta: C**

Pregunta práctica

Complete el código C++ del recorrido inorden:

```
```cpp
void inorden(Nodo* raiz) {
 if (raiz == nullptr) return;
 inorden(raiz->izquierda); // llamada recursividad al subárbol
 izquierdo
 cout << raiz->dato << " ";
 inorden(raiz->derecha); // llamada recursividad al subárbol
 derecho
}
```
```

Respuesta: inorden

2.7 Resultados

2.7.1 Capturas de ejecución en consola

Ejercicio 1: Recorridos manuales del árbol base

2.7.1.1 Ejecución en C++

```
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\Tareas - grupos\Grupo2\tarea2.2 repo_recorridos_arboles_uta> cd src/cpp
>> g++ main.cpp -o recorridos
>> ./recorridos
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
```

2.7.1.2 Ejecución en Java



```
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\Tareas - grupos\Grupo2\tarea2.2-repo_recorridos_arboles_uta> cd src/java
>> javac Main.java
>> java Main
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
```

Ejercicio 2, 3, 4

2.7.1.3 Ejecución en C++

```
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\Tareas - grupos\Grupo2\tarea2.2-repo_recorridos_arboles_uta\src\cpp> g++ main.cpp -o recorridos
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\Tareas - grupos\Grupo2\tarea2.2-repo_recorridos_arboles_uta\src\cpp> .\recorridos

=====
RECORRIDOS DE ARBOLES BINARIOS - UTA
Estudiante: Katherine Moyolema
Curso: Tercero B - Estructura de Datos
=====

--- RECORRIDOS EN PROFUNDIDAD (DFS) ---
Preorden (Raiz -> Izquierda -> Derecha): 10 5 2 1 3 7 15 12 20 18 25 30
Inorden (Izquierda -> Raiz -> Derecha): 1 2 3 5 7 10 12 15 18 20 25 30
Postorden (Izquierda -> Derecha -> Raiz): 1 3 2 7 5 12 18 30 25 20 15 10

--- RECORRIDO EN ANCHURA (BFS) ---
BFS (Nivel por nivel usando cola): 10 5 15 2 7 12 20 1 3 18 25 30

--- ESTADÍSTICAS DEL ÁRBOL ---
Ejercicio 3 - Total de nodos: 12
Ejercicio 4 - Total de hojas: 6
Altura del árbol (extra): 5

--- ANÁLISIS DE RESULTADOS ---
El recorrido Inorden muestra los valores ordenados: 1,2,3,5,7,10,12,15,18,20,25,30
Las hojas del árbol son los nodos: 1, 3, 7, 12, 18, 30
La altura del árbol es 4 (niveles: 0->10, 1->5/15, 2->2/7/12/20, 3->1/3/18/25, 4->30)

=====
FIN DEL PROGRAMA
Los recorridos han sido ejecutados exitosamente
=====
```

2.7.1.4 Ejecución en Java

```
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\Tareas - grupos\Grupo2\tarea2.2-repo_recorridos_arboles_uta\src\java> javac Main.java
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\Tareas - grupos\Grupo2\tarea2.2-repo_recorridos_arboles_uta\src\java> java Main

=====
RECORRIDOS DE ARBOLES BINARIOS - UTA
Curso: Tercero B - Estructura de Datos
=====

--- RECORRIDOS EN PROFUNDIDAD (DFS) ---
Preorden (Raiz -> Izquierda -> Derecha): 10 5 2 1 3 7 15 12 20 18 25 30
Inorden (Izquierda -> Raiz -> Derecha): 1 2 3 5 7 10 12 15 18 20 25 30
Postorden (Izquierda -> Derecha -> Raiz): 1 3 2 7 5 12 18 30 25 20 15 10

--- RECORRIDO EN ANCHURA (BFS) ---
BFS (Nivel por nivel usando cola): 10 5 15 2 7 12 20 1 3 18 25 30

--- ESTADÍSTICAS DEL ÁRBOL ---
Ejercicio 3 - Total de nodos: 12
Ejercicio 4 - Total de hojas: 6
Altura del árbol (extra): 5

--- ANÁLISIS DE RESULTADOS ---
El recorrido Inorden muestra los valores ordenados:
1, 2, 3, 5, 7, 10, 12, 15, 18, 20, 25, 30

Las hojas del árbol son los nodos: 1, 3, 7, 12, 18, 30
Total de hojas: 6

La altura del árbol es 4 (niveles: 0->10, 1->5/15, 2->2/7/12/20, 3->1/3/18/25, 4->30)

=====
FIN DEL PROGRAMA
Los recorridos han sido ejecutados exitosamente
=====
```



2.7.2 Comparación entre lenguajes

Ambos lenguajes de programación (C++ y Java) producen exactamente los mismos resultados algorítmicos al ejecutar los recorridos del árbol binario. Las salidas en consola son idénticas, lo que demuestra que la lógica de los recorridos es independiente del lenguaje de programación utilizado.

- **Sintaxis**
 - C++ usa punteros y `->`
 - Java usa referencias y `.`
- **Memoria**
 - C++ maneja memoria manualmente con `new`
 - Java libera memoria automáticamente con el *Garbage Collector*
- **Nodo**
 - C++: `struct` con punteros
 - Java: `class` con referencias
- **BFS (cola)**
 - C++: `queue<Nodo*>`
 - Java: `Queue<Nodo> = new LinkedList<>()`
- **Recorridos DFS**

Ambos usan recursividad:

 - Preorden
 - Inorden
 - Postorden
- **Funciones adicionales**

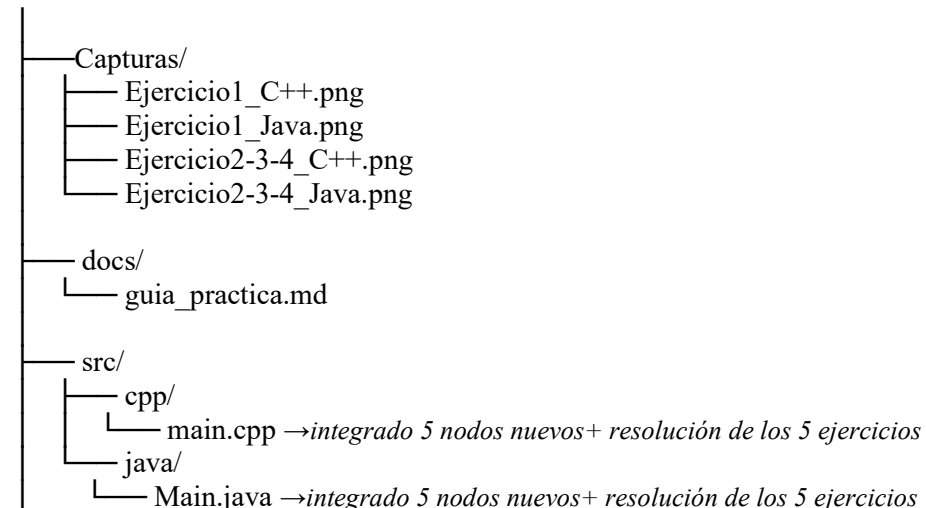
Ambos implementan:

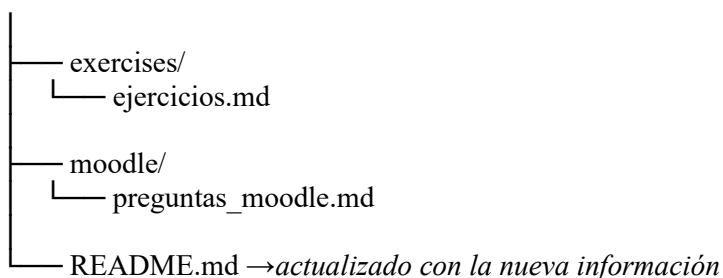
 - contar nodos
 - contar hojas
 - calcular altura
- **Conclusión**
 - C++ → más control y eficiencia
 - Java → más facilidad y manejo automático de memoria
 - Ambos sirven correctamente para implementar árboles binarios y sus recorridos.

2.7.3 Repositorio GitHub

https://github.com/kmoyolema3929/tarea2.2-repo_recorridos_arboles_uta.git

Estructura:





2.8 Conclusiones

- Se implementaron exitosamente los cuatro recorridos de árboles binarios (Preorden, Inorden, Postorden y BFS) en los lenguajes C++ y Java, demostrando que la lógica algorítmica es independiente del lenguaje de programación utilizado. Ambos lenguajes produjeron resultados idénticos en las salidas, validando la correcta comprensión de los conceptos teóricos.
- La recursividad demostró ser una técnica efectiva y elegante para implementar los recorridos DFS (Preorden, Inorden y Postorden), permitiendo aprovechar la naturaleza jerárquica y recursiva de los árboles binarios. Por otro lado, el recorrido BFS requirió obligatoriamente el uso de una cola (Queue) con política FIFO para procesar los nodos nivel por nivel, evidenciando la importancia de seleccionar la estructura de datos adecuada según el tipo de recorrido deseado.
- El caso aplicado del sistema web permitió vincular los conceptos teóricos con una situación real, demostrando que el recorrido Preorden es ideal para mostrar menús jerárquicos, el recorrido Postorden facilita el procesamiento de submódulos antes que el módulo principal (como en cierres de sesión), y el recorrido BFS es óptimo para explorar interfaces por niveles. Esta aplicación práctica refuerza la utilidad de los árboles binarios más allá del ámbito académico, posicionándolos como herramientas fundamentales en el desarrollo de software.

2.9 Recomendaciones

Con base en la experiencia adquirida durante el desarrollo de la práctica, se formulan las siguientes recomendaciones:

- Para futuras implementaciones, se recomienda agregar validaciones que eviten la inserción de nodos duplicados en el árbol, especialmente si se trata de un Árbol Binario de Búsqueda (BST), con el fin de mantener la integridad de los datos y optimizar las operaciones de búsqueda.
- Se sugiere implementar un menú interactivo en consola que permita al usuario elegir qué tipo de recorrido ejecutar, así como insertar nuevos nodos dinámicamente durante la ejecución del programa. Esto mejoraría la experiencia de usuario y facilitaría la comprensión de los recorridos en tiempo real.
- Se recomienda documentar adecuadamente el código utilizando herramientas como Doxygen para C++ o JavaDoc para Java, lo que permitirá generar documentación automática y facilitará el mantenimiento y la reutilización del código en proyectos futuros. Además, se sugiere implementar pruebas unitarias para verificar el correcto funcionamiento de cada recorrido y de las funciones auxiliares (contar nodos, contar hojas, calcular altura).



2.10 Referencias bibliográficas

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Data Structures and Algorithms in Java, 6th ed. Hoboken, NJ, USA: Wiley, 2014.
- [3] B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.
- [4] Y. Langsam, M. J. Augenstein, and A. M. Tenenbaum, Data Structures Using C and C++, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1996.
- [5] J. Bloch, Effective Java, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [6] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [7] Universidad Carlos III de Madrid, "Árboles binarios y recorridos," OpenCourseWare, 2020. [Online]. Available: <http://ocw.uc3m.es/ciencia-computacion/estructuras-de-datos/material-de-clase/temas/tema-3-arboles/tema-3-arboles.pdf>. [Accessed: May 7, 2026].
- [8] GeeksforGeeks, "Tree Traversals (Inorder, Preorder and Postorder)," GeeksforGeeks, 2025. [Online]. Available: <https://www.geeksforgeeks.org/tree-traversals-inorder-preorder-and-postorder/>. [Accessed: May 7, 2026].

2.11 Anexos